

Progetto Sistemi Multicore

Cola Valerio & Federico Cerboni

7 Febbraio 2025

1 Implementazione MPI

1.1 Introduzione

L'approccio impiegato per implementare Sequence in MPI suddivide il numero dei pattern tra i Rank.

La divisione dei pattern è calcolata in modo da distribuire uniformemente i compiti; eventuali eccedenze sono assegnate all'ultimo processo. Per mantenere la coerenza con l'esecuzione sequenziale, dopo che ogni processo ha concluso la fase di "Ricerca" dei pattern assegnati, gli array e le variabili locali vengono progressivamente inviate al processo principale Root, che li unifica gestendo conflitti di valori come nel caso di `pat_found` il quale presentava valori inferiori una volta concluso il programma. Questo processo di unione evita conteggi doppi o mancanti, salvaguardando i risultati complessivi e permettendo un calcolo accurato dei checksum. L'approccio segue un flusso logico che integra comunicazioni point-to-point (`Send`, `Recv`) e collettive (`AllGather`, `Gatherv`), minimizzando sprechi di risorse e massimizzando la scalabilità. Con questo approccio è possibile avere un programma scalabile rispetto al numero e lunghezza dei pattern.

1.2 Codice

Viene generata la sequenza "sequence" solo dal Root e inviata a tutti gli altri Rank con la funzione `MPI_Bcast`.

Ogni Rank:

- Possiede una copia di `pat_matches`.
- Crea una propria copia degli array `seq_matches` e `pat_found`, inizializzati a `NOT_FOUND`.
- Calcola il numero di pattern assegnati, l'indice del primo e dell'ultimo pattern. Se è l'ultimo Rank, gli verranno assegnati i restanti non compresi nella divisione intera.
- Analizza solo i pattern che gli sono stati assegnati in base a `my_first_pattern` e `my_last_pattern`

Alla fine dell'esecuzione, il Root si occuperà del calcolo e la stampa dei checksum, unificando i dati delle variabili locali `seq_matches`, `pat_found` e `pat_matches` di ogni rank.

1.2.1 Gestione di `seq_matches`

I Rank > 0 inviano i propri `seq_matches` al Root con la `MPI_Send`.

```
1  if(rank > 0){
2      MPI_Send(seq_matches, seq_length, MPI_INT, 0, 0, MPI_COMM_WORLD);
3  }
```

Problema:

“Ogni Rank parte con ogni cella di `seq_matches` = -1. Se quindi un pattern che si ripete tra più Rank o se più Rank trovano più pattern che attraversano la stessa cella, `seq_matches` passerà da -

1 a 0 più volte, creando discrepanze rispetto all'esecuzione sequenziale nella quale per ogni posizione dell'array si passa solo una volta da -1 a 0. Ciò causa problemi nel calcolo del checksum finale poiché dopo che il Root ha sommato tutte le celle di `seq_matches` di ogni Rank in un unico array avremo valori inferiori rispetto al sequenziale.”

Il Root crea `local_seq_matches` e vi inserisce man mano i `seq_matches` di ogni altro Rank, sommando i valori al proprio `seq_matches`.

```
1  if (rank == 0) {
2      int *local_seq_matches;
3      local_seq_matches = (int *)malloc( sizeof(int) * seq_length );
4      for (int i = 1; i < size; i++) { // (1)
5          MPI_Recv(local_seq_matches, seq_length, MPI_INT, i, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
6
7          for (unsigned long j = 0; j < seq_length; j++) { // (2)
8              if (seq_matches[j] >= 0 && local_seq_matches[j] >= 0) { // (3)
9                  seq_matches[j] = seq_matches[j] + local_seq_matches[j] + 1;
10             } else {
11                 if(seq_matches[j] == NOT_FOUND){ // (4)
12                     seq_matches[j] = local_seq_matches[j];
13                 }
14             }
15         }
16     }
17 }
18 free(local_seq_matches);
19 }
```

Itera per `i` volte in base al numero dei Rank in ogni iterazione riceverà con la `MPI_Recv` il `seq_matches` del Rank `i` e lo inserisce nel proprio locale ①.

Itera per ogni indice `j` di `seq_matches` e `local_seq_matches` ②:

- Se entrambi i valori sono ≥ 0 , si aggiunge 1 oltre alla somma ③.
- Se il `seq_matches` del Root è `NOT_FOUND`, si imposta al `seq_matches` il valore di `local_seq_matches` ④.

1.2.2 Gestione di `pat_found`

```
1  int send_counts[size]; // (1)
2  send_counts[rank] = my_patterns;
3  int displs[size]; // (2)
4  displs[rank] = my_first_pattern;
5
6  // (3)
7  MPI_Allgather(&my_patterns, 1, MPI_INT, send_counts, 1, MPI_INT, MPI_COMM_WORLD);
8  MPI_Allgather(&my_first_pattern, 1, MPI_INT, displs, 1, MPI_INT, MPI_COMM_WORLD);
9
10 MPI_Barrier( MPI_COMM_WORLD );
11 // (4)
12 MPI_Gatherv(pat_found + my_first_pattern, my_patterns, MPI_UNSIGNED_LONG,
13             pat_found, send_counts, displs, MPI_UNSIGNED_LONG, 0, MPI_COMM_WORLD);
```

Ogni Rank:

- Crea un array `send_counts` per sapere quanti elementi arrivano da ogni processo ①.
- Crea un array `displs` per sapere dove posizionare i dati di ogni processo ②.
- Usa `MPI_Allgather` per far sapere a tutti i processi quanti dati invierà ciascuno, questa azione è necessaria per utilizzare `Gatherv` che ha bisogno degli array `send_counts` e `displs` completi per ogni Rank ③.
- `MPI_Gatherv` raccoglie tutti i dati (`pat_found`) nel processo Root e vengono ricomposti nell'ordine corretto usando le informazioni di `send_counts` e `displs` all'interno di `pat_found` del Root ④. Il `pat_found + my_first_pattern` viene utilizzato per puntare al primo elemento assegnato al Rank

1.2.3 Gestione di `pat_matches`

```
1  int total_pat_matches = 0;
2      MPI_Reduce(&pat_matches, &total_pat_matches, 1, MPI_INT, MPI_SUM, 0, MPI_COMM_WORLD);
3      MPI_Barrier( MPI_COMM_WORLD );
4      if (rank == 0) {
5          pat_matches = total_pat_matches;
6      }
```

Ogni Rank invia al `total_pat_matches` del Root i propri `pat_matches` effettuando una somma mediante `MPI_Reduce`. Successivamente il Root assocerà al proprio `pat_matches` la variabile `total_pat_matches`.

2 Implementazione CUDA

2.1 Introduzione

L'approccio impiegato per implementare Sequence in CUDA assegna un singolo pattern a ciascun thread, concentrando il calcolo principale all'interno del kernel dedicato alla fase di ricerca. I dati necessari sono caricati in memoria globale del Device e, al termine dell'esecuzione, sono trasferiti in memoria host per il calcolo del checksum. Durante l'esecuzione del kernel, ciascun thread verifica di dover effettivamente lavorare ed esegue operazioni atomiche per mantenere la coerenza dei risultati. Questo modello consente di parallelizzare gran parte della computazione, garantendo buone prestazioni. Tuttavia, la scalabilità è limitata dal numero massimo di thread supportato dalla GPU, che costituisce un vincolo hardware. Ad esempio nel caso del Cluster Sapienza la GPU utilizzata è una NVIDIA Quadro RTX 6000 Turing, questo significa che in linea teorica è possibile utilizzare 2,147,483,647 di blocchi ognuno con 1024 thread quindi circa 2.2 trilioni; con i suoi 4608 core CUDA distribuiti su 72 SM la scheda grafica può gestire un totale di circa 147,356 di thread alla volta. Di conseguenza è possibile invocare un kernel con un quantitativo elevato di thread ma la GPU li gestirà schedulandoli e il quantitativo di dati sarà comunque limitato dalle risorse (registri, memoria condivisa, capacità degli SM ecc...).

2.2 Codice

L'host si occupa di generare la sequenza `sequence`, `seq_matches` ecc...

Con `cudaMalloc()` e `cudaMemcpy()` sposta in memoria globale tutte le variabili necessarie per effettuare correttamente la ricerca delle sequenze.

Il kernel sequencer viene invocato con:

```
1 sequencer<<<<ceil(pat_number/1024.0), 1024>>>>(g_seq_length, g_pat_number, g_sequence,
2                                     d_pat_length, d_pattern, g_seq_matches, g_pat_matches, g_pat_found);
3
```

- $\text{ceil}(\text{pat_number}/1024)$ blocchi, in questo modo ogni thread si occuperà di un solo pattern, nel caso ci siano meno di 1024 pattern verrà utilizzato un solo blocco.
- 1024 thread per blocco, questo perché il massimo numero di thread per SM nell'architettura è 1024

Il kernel:

```
1 __global__ void sequencer(unsigned long *g_seq_length, int *g_pat_number, char *g_sequence, unsigned long *d_pat_length,
2                           char **d_pattern, int *g_seq_matches, int *g_pat_matches, unsigned long *g_pat_found) {
3     unsigned long start;
4     int pat;
5     unsigned long lind;
6
7     int tid = blockIdx.x * blockDim.x + threadIdx.x; // (1)
8     if (tid < *g_pat_number) {
9         pat = tid; // (2)
10        for( start=0; start <= *g_seq_length - d_pat_length[pat]; start++) {
11            for( lind=0; lind<d_pat_length[pat]; lind++) {
12                if ( g_sequence[start + lind] != d_pattern[pat][lind] ) break;
13            }
14            if ( lind == d_pat_length[pat] ) {
15                atomicAdd(g_pat_matches, 1); // (3)
16                atomicExch((unsigned long long*)&g_pat_found[pat], (unsigned long long)start); // (4)
17                break;
18            }
19        }
20        if ( g_pat_found[pat] != (unsigned long)NOT_FOUND ) {
21            increment_matches( pat, g_pat_found, d_pat_length, g_seq_matches );
22        }
23    }
24    __syncthreads();
25 }
26 __device__ void increment_matches( int pat, unsigned long *pat_found, unsigned long *pat_length, int *seq_matches ) {
27     unsigned long ind;
28     for( ind=0; ind<pat_length[pat]; ind++) {
29         atomicAdd(&seq_matches[ pat_found[pat] + ind ], 1); // (5)
30     }
31 }
```

Verifica il numero del thread e che sia uno di quelli che devono lavorare indipendentemente dal blocco ①, successivamente utilizza il suo indice come variabile pat in modo che coincida con l'indice del pattern con cui deve lavorare ②. È necessario l'utilizzo di operazioni atomiche:

- Somma per incrementare g_pat_matches (pat_matches) ③.
- Scambio per assegnare start a g_pat_found[pat] ④.
- Somma per incrementare g_seq_matches (seq_matches) all'interno della funzione increment_matches ⑤. È stato anche necessario modificare questa funzione, è stato lasciato solo il corpo dell'else, poichè alla fine dell'esecuzione l'array seq_matches presentava valori inferiori.

Dopo l'esecuzione del kernel vengono riportati in memoria host solo i dati necessari al checksum e viene liberata la memoria con cudaFree(). Si occuperà alla fine l'host di calcolare i checksum e stamparli.

3 Implementazione MPI + CUDA

3.1 Introduzione

L'approccio utilizzato per implementare Sequence in MPI+CUDA suddivide equamente il numero di pattern tra i Device, e ogni processo MPI gestisce una GPU. Anche in questo caso se la suddivisione non è intera, i pattern in eccesso vengono assegnati all'ultimo Rank/Device. Il codice non si limita all'utilizzo di due GPU, ma può essere eseguito su un numero maggiore di dispositivi. Ogni Processo alloca localmente i dati necessari ai calcoli e li invia al proprio Device, dove il kernel, in modo simile all'implementazione CUDA, esegue la ricerca sui dati esclusivamente sulla porzione decisa durante la spartizione.

Al termine, come nella versione MPI, i dati per il checksum vengono trasferiti nella memoria host di ciascun rank, e il processo Root unifica i risultati di ogni Processo nelle proprie variabili locali con le stesse modalità discusse per l'implementazione MPI. Siamo a conoscenza che per quanto riguarda la memoria non è molto efficiente allocare ad ogni GPU ad esempio tutto l'array `d_path_length` poichè lavorerà solo su una parte di esso.

3.2 Codice

La versione MPI+CUDA estende il codice base della versione CUDA aggiungendo le funzioni di inizializzazione MPI all'inizio del main e utilizzando la variabile rank come argomento di `cudaSetDevice()`.

Solo il Root genera la sequenza e la trasmette a tutti i processi con Bcast.

Nel medesimo modo della versione MPI ogni processo:

- Alloca copie locali delle variabili.
- Calcola quanti pattern elaborare (se è l'ultimo rank include quelle rimanenti).
- Trasferisce i dati sul proprio Device con `cudaMalloc` e `cudaMemcpy`.

```
1  __global__ void sequencer(unsigned long *g_seq_length, int *g_my_pat_number, char *g_sequence, unsigned long *d_pat_length,
2                               char **d_pattern, int *g_seq_matches, int *g_pat_matches, unsigned long *g_pat_found, int *g_my_first_pattern) {
3      unsigned long start;
4      int pat;
5      unsigned long lind;
6
7      int tid = blockIdx.x * blockDim.x + threadIdx.x;
8      if (tid < *g_my_pat_number) { // (1)
9          pat = tid + *g_my_first_pattern; // (2)
10         for( start = 0; start <= *g_seq_length - d_pat_length[pat]; start++) {
11             for( lind=0; lind < d_pat_length[pat]; lind++) {
12                 if ( g_sequence[start + lind] != d_pattern[pat][lind] ) break;
13             }
14             if ( lind == d_pat_length[pat] ) {
15                 atomicAdd(g_pat_matches, 1);
16                 atomicExch((unsigned long *)&g_pat_found[pat], (unsigned long)start);
17                 break;
18             }
19         }
20         if ( g_pat_found[pat] != (unsigned long)NOT_FOUND ) {
21             increment_matches( pat, g_pat_found, d_pat_length, g_seq_matches );
22         }
23     }
24 }
```

- Invoca il kernel `sequencer`, che si differenzia dalla versione CUDA, passando `g_my_pat_number` e `g_my_first_pattern`, così ciascun thread lavora sui pattern corrispondenti al proprio intervallo. Viene effettuata una verifica per far lavorare solo il numero di thread necessari ① e per allinearsi al numero di pattern corretto vengono sommati il numero di thread e `g_my_first_pattern` ②.

Se ad esempio ad ogni Device sono assegnati 100 pattern, il thread 5 del secondo Device lavora sul pattern $5 + 100 = 105$ poiché i primi 100 pattern spettano al primo.

Al termine, i dati per il checksum sono copiati nell'host e la memoria del Device viene liberata. Il processo di unificazione e calcolo dei checksum è identico alla versione MPI.

4 Implementazione OpenMP

In questa implementazione la regione di codice che è stata parallelizzata è quello della ricerca dei pattern.

Per definire il blocco di codice parallelo è stata utilizzata la direttiva pragma per condividere tra tutti i thread gli array `pat_found` e `pat_matches` ①.

Solo il primo ciclo for è stato parallelizzato poiché, nonostante i nostri tentativi, non siamo riusciti a parallelizzare anche i cicli annidati. Per

utilizzare la clausola `collapse()` è necessario che i cicli siano perfettamente annidati senza istruzioni intermedie e non contengano `break` che interrompono i cicli. Inoltre, i domini degli ultimi due for si intersecano poiché la variabile `pat` viene utilizzata come condizione sia per questi cicli che come variabile di controllo del primo for. Questa dipendenza impedisce l'applicazione efficace della clausola `collapse()`, che richiede domini indipendenti e senza interferenze.

Sono state utilizzate due direttive pragma per rendere il codice più performante, notiamo come nella prima viene inizializzato il team di processi e successivamente viene distribuito il lavoro per il for. In questo modo diminuiamo l'overhead di creazione dei processi inizializzandoli un'unica volta. Assieme all'omp for sono state rese private le variabili di controllo di ogni ciclo for, è stata utilizzata una reduction per effettuare la somma di `pat_matches` alla fine del blocco parallelo poiché ogni thread calcola un determinato numero di pattern e incrementa privatamente quando ne trova uno ②.

Perché non utilizzare `firstprivate()`?

Il codice avrebbe in entrambi i casi lo stesso comportamento perché non abbiamo bisogno che ogni processo abbia il valore della variabile inizializzata al di fuori del blocco in quanto sono tutte necessarie ai controlli dei cicli e vengono inizializzate di nuovo da ogni processo.

Lo schedule utilizzato è `dynamic` poiché nei test le sequenze hanno lunghezza variabile, di conseguenza ogni thread potrebbe concludere il lavoro con tempistiche diverse.

In conclusione mediante la sezione critica si protegge l'accesso concorrente alla funzione `increment_matches`, che si occupa di modificare `seq_matches`, la quale è fondamentale per evitare che nell'array ci siano valori inferiori alla fine dell'esecuzione ③.

```
1 unsigned long start;
2 int pat;
3 #pragma omp parallel shared(pat_found, seq_matches) // (1)
4 {
5     // (2)
6     #pragma omp for private(start, pat, lind) \
7     reduction(+:pat_matches) schedule(dynamic,4)
8
9     for( pat=0; pat < pat_number; pat++ ) {
10         for( start=0; start <= seq_length - pat_length[pat]; start++) {
11             for( lind=0; lind<pat_length[pat]; lind++) {
12                 if ( sequence[start + lind] != pattern[pat][lind] ) break;
13             }
14             if ( lind == pat_length[pat] ) {
15                 pat_matches++;
16                 pat_found[pat] = start;
17                 break;
18             }
19         }
20         if ( pat_found[pat] != (unsigned long)NOT_FOUND ) {
21             #pragma omp critical // (3)
22             increment_matches( pat, pat_found, pat_length, seq_matches );
23         }
24     }
25 }
```

5 Testing e Efficienza

5.1 Introduzione

Per l'esecuzione dei test è stato realizzato uno script che ripete l'esecuzione di tutte le versioni del programma, con diversi argomenti in input, in modo da verificare diverse situazioni con complessità e quantità di dati crescenti.

Le tipologie di test effettuati sono i seguenti:

1. Test dove aumenta la lunghezza della sequenza
2. Test dove aumenta il numero di pattern da cercare
3. Test dove aumentano la lunghezza della sequenza, il numero di pattern e la lunghezza dei pattern stessi.

Non verranno mostrati test dove aumenta solo la grandezza dei pattern, perché nella nostra implementazione il parallelismo non scala con l'aumentare di tale grandezza (i tempi rimangono sempre uguali).

5.1.1 MPI

Con input di piccola dimensione, l'overhead incide pesantemente e riduce le performance del programma rispetto alla versione sequenziale, ma con input di media e grande dimensione, i costi di comunicazione vengono compensati dalla maggiore parallelizzazione, e quindi la velocità aumenta e cresce linearmente all'aumentare del numero dei processi.

Questo è grazie al fatto che la parallelizzazione con MPI risulta ottimale per questo tipo di programma, in quanto i pattern da cercare possono essere suddivisi in maniera equa ai vari processi, ed essi cercheranno tali pattern in maniera completamente indipendente l'uno dall'altro. La comunicazione (overhead) avviene soltanto all'inizio e alla fine dell'esecuzione.

5.1.2 OpenMP

OpenMP in generale, confrontandolo con MPI a parità di processi/thread, risulta sempre più lento di quest'ultimo nei test più piccoli, ma con input più grandi, tranne nei test del numero di pattern, diventa sempre più veloce. Questo è dovuto al fatto che OpenMP si basa sul paradigma di parallelizzazione a memoria condivisa, quindi a differenza di MPI, non introduce un significativo overhead di comunicazione. OpenMP mostra i suoi punti deboli solamente nei casi in cui si verificano molti conflitti di memoria, ovvero quando gli input dei test sono piccoli, e anche nel caso di un numero di pattern molto alto da cercare.

4.1.3 CUDA

Il test dove ci sono molti pattern, CUDA è molto più veloce per 2 motivi principali:

1. tanti pattern da cercare corrispondono a tanti thread in esecuzione sulla gpu, quindi viene sfruttato l'alto parallelismo della gpu (occupancy).
2. I thread quando cercano i pattern nelle sequenze, fanno tanti accessi in memoria, se un warp va in attesa per l'accesso in memoria lo scheduler assegna nuovi thread in modo che gli SM siano sempre occupati.

Invece quando i pattern sono pochi, l'occupancy della gpu è molto bassa, quindi non andiamo a sfruttare il parallelismo della gpu. Questo perché tutti i thread vengono assegnati a pochi SM, quindi la GPU non viene utilizzata nella sua completezza.

Se concettualmente andassimo a distribuire questi pochi pattern su tutti gli SM, essi sarebbero comunque per la maggior parte del tempo non attivi, a causa dell'attesa degli accessi in memoria, durante i quali lo scheduler non ha altri thread da sostituire.

In questo caso le CPU sono migliori, siccome in un processore (core) viene eseguito un thread alla volta, il processore avrà sicuramente un thread da sostituire, quindi meno tempo di attesa, anche se più overhead. Inoltre con pattern molto lunghi, un singolo thread eseguirà molti più accessi in memoria, e avrà bisogno di molta più memoria cache, cosa che le classiche CPU hanno di più rispetto alle GPU (cache L1).

5.1.4 MPI + CUDA

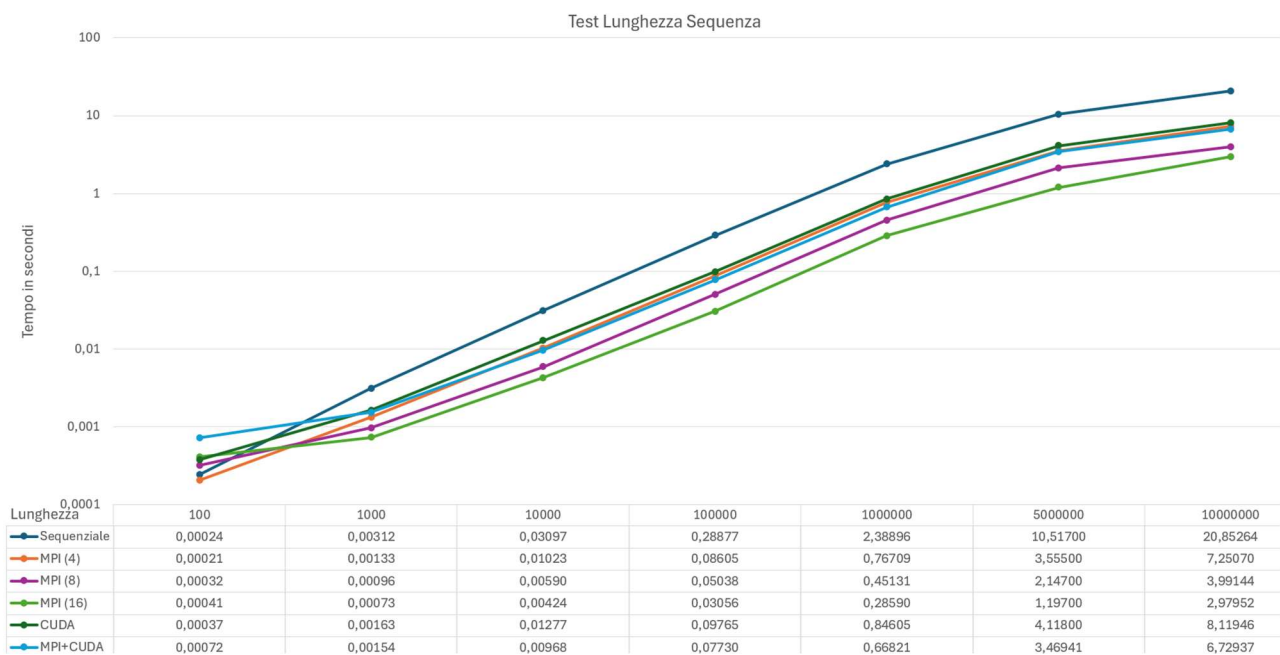
Le stesse cose si applicano ovviamente anche a MPI + CUDA, in quanto la computazione viene effettuata sempre dalle GPU. Infatti il vantaggio di usare due GPU in parallelo si vede solo in quei test che hanno tantissimi pattern da ricercare. Altrimenti il vantaggio di avere due GPU in parallelo rispetto alla versione CUDA normale viene meno per 2 motivi: per l'overhead introdotto da MPI, e soprattutto perché, siccome ogni GPU avrà la metà dei pattern da verificare, ovvero la metà dei thread da eseguire, l'occupancy diminuisce, soprattutto quando i pattern in partenza sono pochi, per gli stessi motivi precedenti.

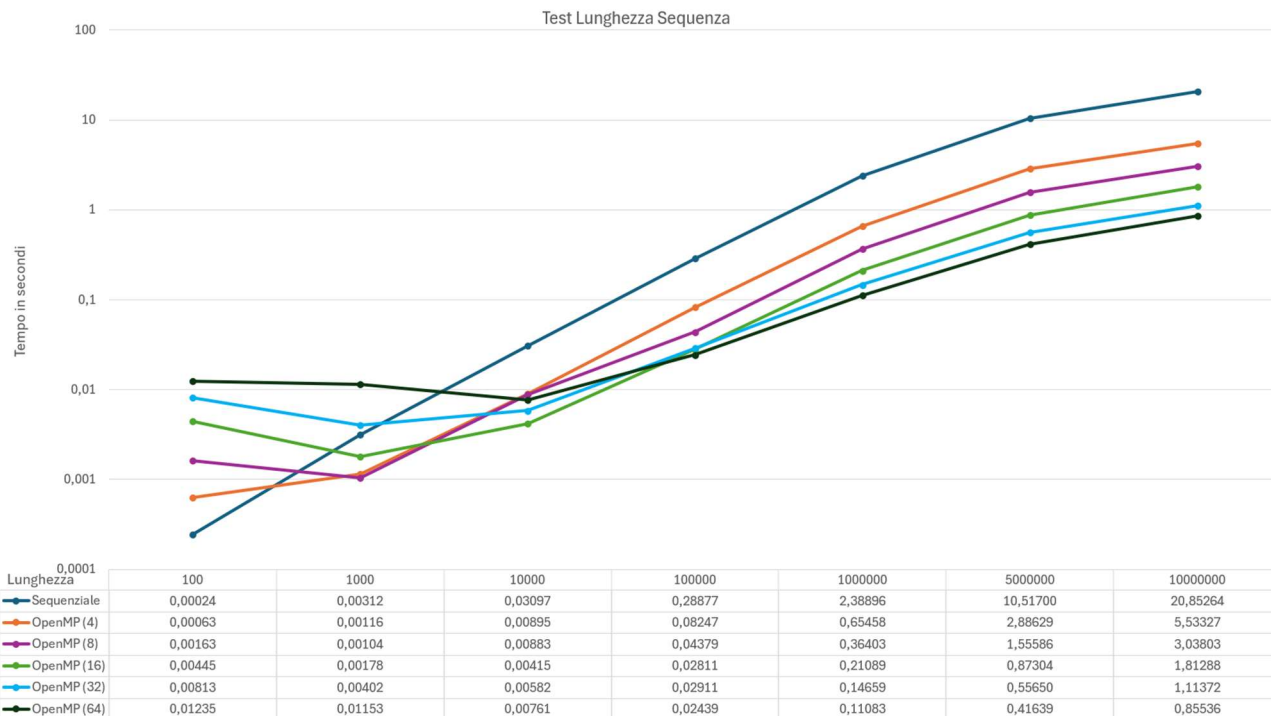
5.2 Fase di test

Ogni dato presente nelle seguenti tabelle è stato ottenuto effettuando la media dei migliori risultati per ogni test. Inoltre per il calcolo di efficienza e speedup e per verificare se i nostri programmi siano weak/strong scalable abbiamo preso in considerazione solo il test nel quale vengono incrementate il numero di sequenze in quanto tutte le nostre implementazioni si basano sul suddividere il numero di Pattern tra i processi.

5.2.1 Test lunghezza sequenza DNA

In questo test, le versioni parallele del codice nei test più piccoli sono più veloci della versione sequenziale del programma. Dal secondo test MPI e OpenMP diventano più veloci e scalano con l'aumentare del numero di processi/thread. MPI con 32 processi risulta sempre peggiore rispetto alla versione con 16 processi, mostrando i sintomi di un overhead di comunicazione eccessivo. MPI inizialmente (test 2 e 3) risulta più veloce di OpenMP, ma nei test più grandi quest'ultimo diventa più veloce.

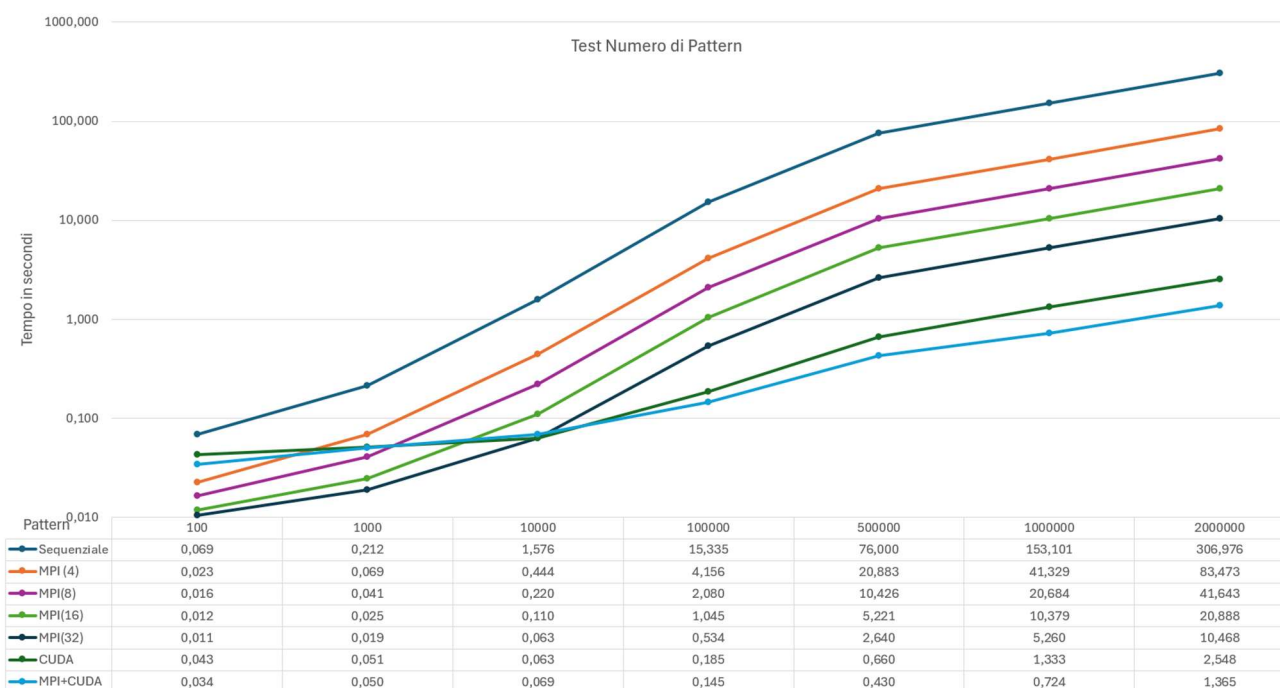


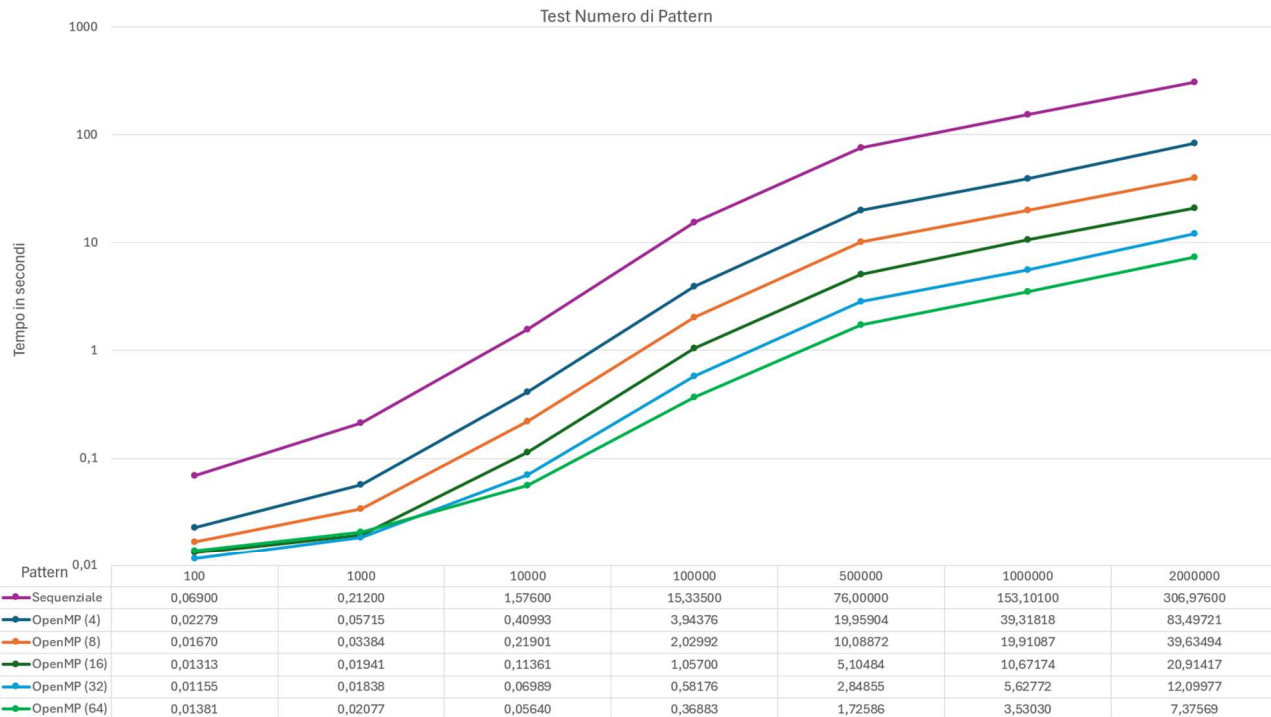


4.2.2 Test numero pattern

In questo test MPI e OpenMP si mostrano subito più veloci del sequenziale, e i tempi di esecuzione diminuiscono linearmente all'aumentare del numero di processi/thread in esecuzione. Nei primi 3 test i tempi di esecuzione di MPI e OpenMP sono simili, ma aumentando la grandezza di input, MPI scala in maniera quasi perfettamente lineare, a differenza di OpenMP, soprattutto nei test con un grande numero di thread.

CUDA e MPI+CUDA inizialmente sono più lenti di MPI e OpenMP, ma all'aumentare del numero di pattern in input, la velocità di esecuzione diventa superiore rispetto a tutte le altre versioni, con MPI+CUDA che raggiunge la metà del tempo di esecuzione rispetto a CUDA nei test più pesanti, mostrando un'eccellente scalabilità.





	100	1000	10000	100000	500000	1000000	2000000
MPI (4)	S: 3,00, E: 0,75	S: 3,07, E: 0,77	S: 3,55, E: 0,89	S: 3,69, E: 0,92	S: 3,64, E: 0,91	S: 3,70, E: 0,93	S: 3,68, E: 0,92
MPI (8)	S: 4,31, E: 0,54	S: 5,17, E: 0,65	S: 7,16, E: 0,89	S: 7,37, E: 0,92	S: 7,29, E: 0,91	S: 7,40, E: 0,93	S: 7,37, E: 0,92
MPI (16)	S: 5,75, E: 0,36	S: 8,48, E: 0,53	S: 14,33, E: 0,90	S: 14,67, E: 0,92	S: 14,56, E: 0,91	S: 14,75, E: 0,92	S: 14,69, E: 0,92
MPI (32)	S: 6,27, E: 0,20	S: 11,16, E: 0,35	S: 25,02, E: 0,78	S: 28,72, E: 0,90	S: 28,79, E: 0,90	S: 29,10, E: 0,91	S: 29,32, E: 0,92
OpenMP (4)	S: 3,03, E: 0,76	S: 3,71, E: 0,93	S: 3,84, E: 0,96	S: 3,89, E: 0,97	S: 3,81, E: 0,95	S: 3,89, E: 0,97	S: 3,68, E: 0,92
OpenMP (8)	S: 4,13, E: 0,52	S: 6,27, E: 0,78	S: 7,20, E: 0,90	S: 7,56, E: 0,95	S: 7,53, E: 0,94	S: 7,69, E: 0,96	S: 7,75, E: 0,97
OpenMP (16)	S: 5,25, E: 0,33	S: 10,93, E: 0,68	S: 13,87, E: 0,87	S: 14,51, E: 0,91	S: 14,88, E: 0,93	S: 14,35, E: 0,90	S: 14,68, E: 0,92
OpenMP (32)	S: 5,97, E: 0,19	S: 11,53, E: 0,36	S: 22,56, E: 0,70	S: 26,35, E: 0,82	S: 26,68, E: 0,83	S: 27,22, E: 0,85	S: 25,39, E: 0,79
OpenMP (64)	S: 5,00, E: 0,08	S: 10,20, E: 0,16	S: 27,94, E: 0,44	S: 41,56, E: 0,65	S: 44,05, E: 0,69	S: 43,39, E: 0,68	S: 41,60, E: 0,65
CUDA	S: 1,60 E: 0,022	S: 4,16 E: 0,058	S: 25,02 E: 0,35	S: 82,81 E: 1,15	S: 115,15 E: 1,6	S: 114,87 E: 1,6	S: 120,52 E: 1,67
MPI+CUDA	S: 2,03 E: 0,014	S: 4,24 E: 0,029	S: 22,83 E: 0,158	S: 105,41 E: 0,732	S: 176,74 E: 1,227	S: 211,52 E: 1,468	S: 224,88 E: 1,56

Speedup:

È possibile notare come MPI mantenga un buon Speedup con la versione da 32 processi che passa dall'essere 6 volte più veloce rispetto al sequenziale per problemi più piccoli fino a 29 volte più veloce per problemi molto più complessi con 4 milioni di sequenze da ricercare.

Per quanto riguarda OpenMP il comportamento è simile mantenendo da 4 a 16 processi uno speedup mediamente costante tralasciando il test più piccolo. Invece per le versioni da 32 e 64 notiamo un importante incremento rispettivamente da 6 a 25 volte più veloce e da 5 a 41.

Per CUDA e MPI+CUDA notiamo un notevole incremento in quanto si passa rispettivamente da 1,6 a 120,5 volte più veloce e da 2 a 224.

Efficienza:

È possibile vedere come sia MPI che OpenMP mantengano una buona efficienza tendente a 1 in tutte le versioni con test di complessità medio elevata, con i test più piccoli invece notiamo come vada a diminuire con l'aumentare dei processi questo poiché la complessità è molto piccola ed aumentando il numero di processi si crea maggiore overhead, il quale invece viene coperto nel caso di problemi con una complessità elevata.

Per CUDA e MPI+CUDA nei test meno complessi è estremamente bassa perché l'occupancy risulta molto bassa, con i test più grandi aumenta drasticamente oltre l'1. Ciò succede perché sfruttiamo il latency hiding, possibile solo nelle GPU a differenza di MPI e OpenMP. MPI+CUDA inoltre risulta

meno efficiente rispetto a CUDA pur avendo un minore tempo di esecuzione, ciò a causa dell'overhead introdotto da MPI.

Strong Scaling:

Con input piccoli lo strong scaling è pessimo, perché l'overhead introdotto dalle implementazioni non viene compensato dal tempo di esecuzione, mentre con input grandi possiamo notare come MPI abbia uno strong scaling molto buono, dato che l'efficienza rimane sempre la stessa all'aumentare del numero di processi, mentre con OpenMP con un alto numero di thread, lo strong scaling non è altrettanto buono e l'efficienza peggiora. Questo perché l'alto numero di thread che hanno accesso concorrentiale alla memoria condivisa andranno incontro ad un non trascurabile numero di conflitti di memoria.

Weak Scaling:

Quando arriviamo al massimo della saturazione del processore, con 32 processi MPI e 64 thread OpenMP (che corrispondono al numero di cores/thread del processore sul quale sono stati eseguiti i test), il weak scaling peggiora, a causa del fatto che ci sono in esecuzione sulla macchina anche altri processi (Sistema Operativo, servizi necessari per il funzionamento della macchina) che interferiscono con l'esecuzione dei programmi. Negli altri casi, il weak scaling sia per MPI che per OpenMP risulta molto buono.

Occupancy:

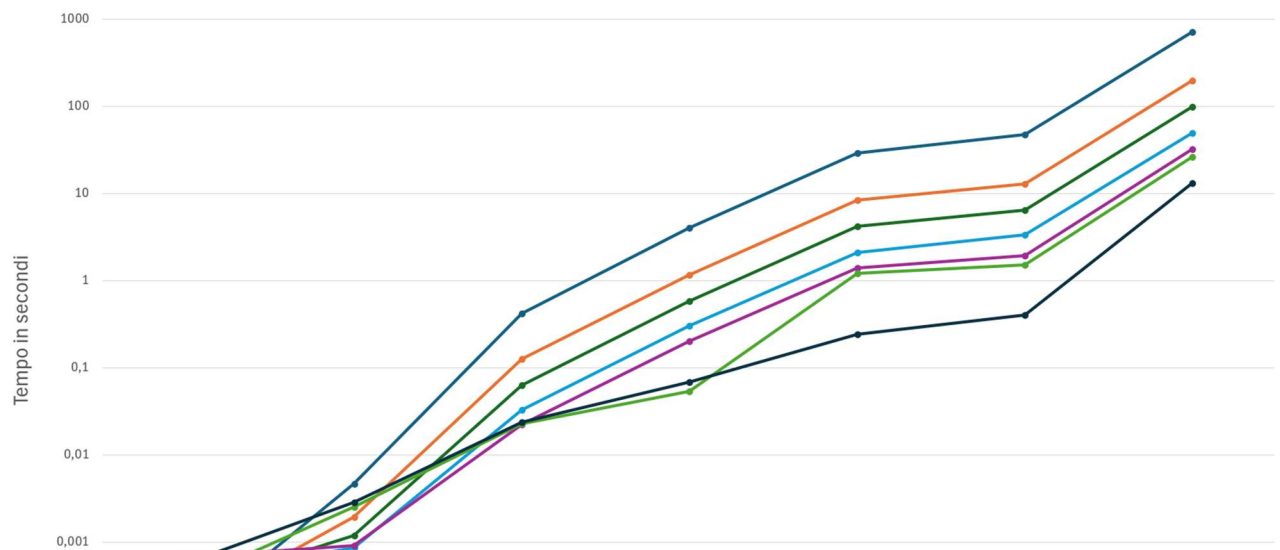
Per effettuare questi test è stata utilizzata la Tesla T4 (40 SM) di Google Colab.

Utilizzando il profiler Nsight Compute abbiamo verificato l'occupancy della GPU variando la dimensione dei blocchi. Un primo test con 1 milione di pattern e 1024 thread per blocco si hanno 977 blocchi, quindi assegnabili circa 24 blocchi per SM, andando oltre la capacità di 16 blocchi residenti per SM. Siccome ogni blocco è da 1024 thread abbiamo utilizzato la capacità massima di ogni SM, quindi l'occupancy attesa è del 100% e corrisponde a quella misurata dal profiler. Con un secondo test da 32 thread per blocco con stesso input si hanno 31250 blocchi e un massimo di 781 blocchi per SM. Logicamente ne andranno assegnati sempre 16 blocchi, siccome ognuno è da 32 thread, utilizziamo 512 thread per SM, la metà del massimo. L'occupancy attesa si dovrebbe aggirare sul 50%, infatti quella misurata è del 48,92%. Ciò significa che non ci sono ulteriori limitazioni oltre alla dimensione dei blocchi.

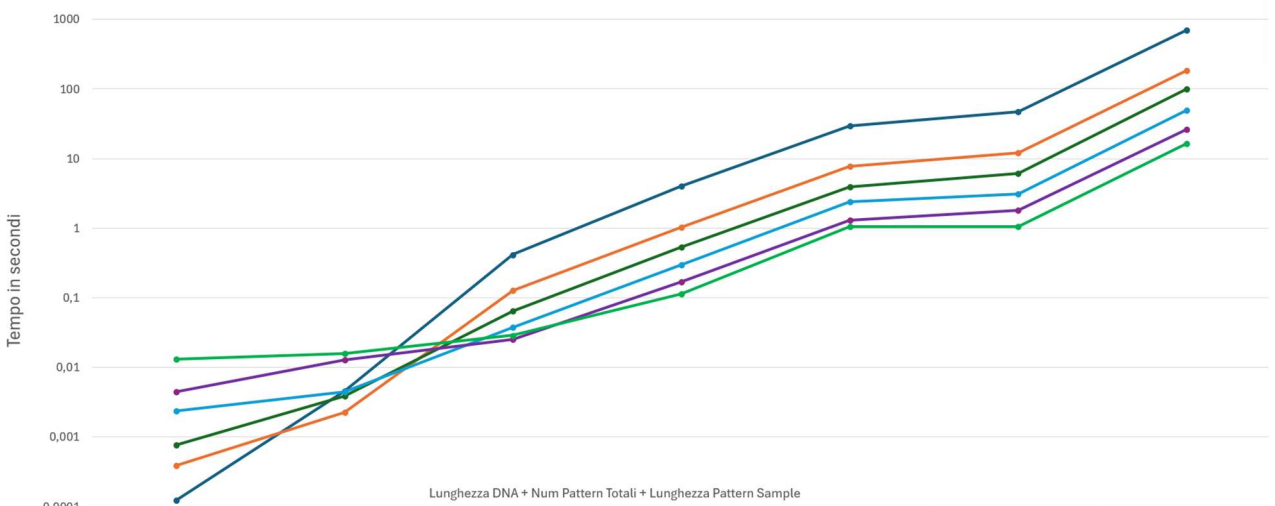
Il nostro kernel utilizza 22 registri, con 1024 thread assegnati per SM utilizzeremo 22528 registri per SM, quindi sotto il limite massimo di 65536 registri per SM. Ciò significa che il numero di registri utilizzati non andrà a limitare l'occupancy e il latency hiding.

4.2.3 Test Complessi

Il seguente test è stato scelto per studiare il comportamento dei programmi in casi complessi, unendo le caratteristiche dei test precedenti. Si può osservare che con gli input più piccoli le versioni parallele sono più veloci della versione sequenziale. Confrontando MPI e OpenMP si può notare che si comportano allo stesso modo del test lunghezza sequenza, con MPI inizialmente più veloce ma con OpenMP che prevale nei test più complessi. CUDA e MPI+CUDA diventano i più veloci rispettivamente dal test 4 e dal test 5, dove si può notare che in alcuni test, le performance non scalano linearmente, con CUDA che risulta molto più veloce nel test 4 e MPI+CUDA nei test 5 e 6.



	Lunghezza DNA + Num Pattern Totali + Lunghezza Pattern Sample						
	100 + 600 + 100	1000 + 1500 + 1000	10000 + 10500 + 10000	30000 + 30500 + 10000	80000 + 80500 + 20000	100000 + 100500 + 100000	500000 + 300500 + 300000
Sequenziale	0,00012	0,00458	0,41785	4,04427	29,16932	47,56545	706,92578
MPI (4)	0,00017	0,00194	0,12437	1,16123	8,30514	12,87930	195,43698
MPI (8)	0,00029	0,00117	0,06326	0,58345	4,18895	6,44594	97,55554
MPI (16)	0,00041	0,00086	0,03287	0,29929	2,10466	3,31175	49,13799
MPI (32)	0,00067	0,00091	0,02200	0,19921	1,39009	1,93267	32,22498
CUDA	0,00035	0,00253	0,02244	0,05315	1,21745	1,50090	26,11698
MPI+CUDA	0,00057	0,00283	0,02353	0,06789	0,24337	0,40097	13,07969



	Lunghezza DNA + Num Pattern Totali + Lunghezza Pattern Sample						
	100 + 600 + 100	1000 + 1500 + 1000	10000 + 10500 + 10000	30000 + 30500 + 10000	80000 + 80500 + 20000	100000 + 100500 + 100000	500000 + 300500 + 300000
Sequenziale	0,00012	0,00458	0,41785	4,04427	29,16932	47,56545	706,92578
OpenMP (4)	0,00039	0,00229	0,12929	1,04016	7,66988	12,00523	181,91509
OpenMP (8)	0,00077	0,00384	0,06482	0,53388	3,88950	6,11382	99,96986
OpenMP (16)	0,00239	0,00447	0,03766	0,29724	2,41949	3,12135	48,92357
OpenMP (32)	0,00453	0,01283	0,02529	0,16821	1,31652	1,80053	26,54257
OpenMP (64)	0,01305	0,01585	0,02883	0,11347	1,04394	1,05810	16,26776